



**UNIVERSIDADE FEDERAL
DE SANTA CATARINA**

Centro Tecnológico - Departamento de Informática e Estatística

Ciências da Computação

INE5424 - Sistemas Operacionais II

Relatório de projeto

Comunicação em Sistemas Autônomos Críticos

Entrega 4 de 7

Anderson Simioni Assunção (23150566)

Bruno Pavani Bertolino (23100464)

Gustavo Fukushima de Salles (21250090)

Julia Fischer Gazolla (23250586)

Professores: Antonio Augusto Medeiros Frohlich e Odorico Machado Mendizabal

20 de outubro de 2025

ÍNDICE

1. INTRODUÇÃO

- 1.1. Requisitos globais
- 1.2. Desafio da etapa 1
- 1.3. Desafio da etapa 2
- 1.4. Desafio da etapa 3
- 1.5. Desafio da etapa 4

2. DESENVOLVIMENTO DA ETAPA 1

- 2.1. Makefile e run_vms.sh
- 2.2. main.cpp e gateway.h
- 2.3. ethernet.h
- 2.4. nic.h e nic.cpp
- 2.5. protocol.h
- 2.6. communicator.h e channel_origin.h
- 2.7. Diagramas de sequência de recebimento de mensagem
- 2.8. car_component.h e components.h
- 2.9. engine.h, engine_ethernet.h e engine_shm.h
- 2.10. Testes de comunicação entre duas máquinas virtuais
- 2.11. Avaliação simples de desempenho

3. DESENVOLVIMENTO DA ETAPA 2

- 3.1. gateway.h
- 3.2. car.h e car.cpp
- 3.3. shm_node.h e shm_node.cpp

4. DESENVOLVIMENTO DA ETAPA 3

- 4.1. Sinal da Kernel

5. DESENVOLVIMENTO DA ETAPA 4

6. CONCLUSÃO

- 6.1. Dificuldades anteriores
- 6.2. Conclusão final após etapa 4

1. INTRODUÇÃO

O presente documento faz parte da entrega parcial da Proposta de Projeto para INE5424 - Sistemas Operacionais II em 2025/2 intitulado Comunicação em Sistemas Autônomos Críticos. A proposta gira em torno do desenvolvimento de uma biblioteca de comunicação confiável e segura para sistemas autônomos críticos, ambientada no cenário de veículos autônomos.

O projeto será dividido em sete etapas, sendo que em cada uma delas a equipe terá desafios. As soluções propostas pela equipe serão apresentadas em relatório e em apresentação em sala, com slides.

1.1 REQUISITOS GLOBAIS

Dentre os requisitos globais do projeto, temos:

- O desenvolvimento deve ser feito na linguagem de programação C++ utilizando apenas a C Standard Library (libc) e a C++ Standard Library em plataforma POSIX nativa.
- Cada sistema autônomo (e.g. veículo) deve ser modelado como um macro-objeto que tem associado a si uma máquina virtual (VM) QEMU.
- Cada componente de cada um dos sistemas autônomos (e.g. sensor, fusor, modelo de ML) deve ser modelado como um macro-objeto que tem associado a si um processo do sistema operacional virtualizado.
- A comunicação via rede se dará sempre por broadcast no nível físico, com alcance limitado pelo domínio de colisão inerente da respectiva tecnologia (uma célula de rádio 5G ou uma rede local Ethernet). Portanto, domínios de colisão serão modelados simplesmente pela conexão de VMs em redes virtuais (VLANs) distintas.
- A biblioteca de comunicação deve apresentar uma API unificada para todos os agentes, independente de serem sistemas autônomos ou componentes dos mesmos. As mensagens trocadas entre os agentes têm tamanho máximo conhecido e menor do que a MTU da rede, logo, nunca serão fragmentadas.
- Os eventos assíncronos da pilha de protocolos de comunicação em desenvolvimento neste projeto devem ser tratados pelo prisma do padrão de projeto Observer X Observed, com as entidades observadas notificando seus observadores para que atualizem seus estados. Em particular, a recepção de mensagens deve ser sempre assíncrona, sem protocolos de rendezvous, e os eventos de recepção de pacotes pelo kernel do SO devem ser imediatamente propagados à pilha de protocolos. Essa propagação pode se dar tanto através da implementação de módulos específicos do protocolo para o kernel quando através de sinais.
- Os grupos serão constituídos por até 4 alunos e podem incluir alunos de ambas as turmas desde que os mesmos estejam presentes em todas as apresentações de andamento do projeto, em todos os atendimentos extraclasse e também na apresentação final.
- Depois de desenvolver e testar cada etapa do projeto, os grupos submeterão, via Moodle, um link para um commit específico em seu próprio repositório Git, com

todas as especificações e diagramas de projeto localizados em uma pasta chamada “doc”. Junto com o link, os grupos enviarão as credenciais de acesso para que os avaliadores possam acessar o código (repositórios abertos não podem ser utilizados; GitHub e <https://codigos.ufsc.br/> são boas alternativas). Na raiz da árvore de desenvolvimento, deve existir um Makefile capaz de acionar a compilação e a execução de todos os testes pertinentes a avaliação simplesmente com o comando make.

- Os slides usados nas apresentações de andamento de cada uma das etapas do projeto podem tanto estar na mesma pasta “doc” no Git ou em um documento online referenciado pela segunda URL da tarefa no Moodle. As apresentações devem conter uma avaliação simples de desempenho com latência média observada durante os testes.

1.2 DESAFIO DA ETAPA 1 - Preparação do Cenário de Comunicação entre Sistemas Autônomos

Nesta etapa, uma versão minimalista da API deve ser implementada para suportar a modelagem do cenário de comunicação entre sistemas autônomos que sustentará as demais etapas. A comunicação entre sistemas autônomos, ou seja, entre as VMs que abstraem veículos, pode ser feita com raw sockets para a transferência de frames Ethernet (sem IP) através da classe Engine referenciada na especificação da API. O requisito de propagação imediata de mensagens não precisa ser considerado nesta etapa e implementações de módulos de kernel não devem ser considerados. Os testes devem modelar ao menos 5 veículos implementados como VMs QEMU e cujos componentes (e.g. sensores, powertrain, ECUs) devem ser implementados como processos POSIX. As VMs devem se comunicar via uma rede virtual privada.

1.2.1 Mensagens

Nesta etapa, as mensagens são um simples array de bytes encapsulados em um frame Ethernet como payload:

$M = \{.*\}$

1.2.2 Identificadores

A plena identificação dos atores da comunicação é fundamental para o correto funcionamento da biblioteca sendo desenvolvida e pode ser feita de muitas formas diferentes. Os endereços na classe NIC (i.e. `NIC::Address`) são endereços Ethernet (a.k.a. MAC Address), e podem ser utilizados para identificar as VMs, mas é importante lembrar que uma mesma máquina, com mais de uma placa de rede, terá mais de um desses endereços. Além disso, é importante ter-se em mente que os requisitos do projeto exigem que o endereço destino na camada de enlace seja o de broadcast (`FF:FF:FF:FF:FF:FF`).

1.3 DESAFIO DA ETAPA 2 - Comunicação entre Componentes de um mesmo Sistema Autônomo

Nesta etapa, a API implementada na etapa anterior para a comunicação entre sistemas autônomos deve ser estendida para suportar a comunicação entre os componentes de um mesmo sistema, ou seja, entre processos rodando em uma mesma VM. Toda a diferenciação

deve ser confinada à classe Engine referenciada na especificação da API, sendo que esta segunda Engine deve ser implementada usando memória compartilhada em um modelo Produtor X Consumidor no contexto de IPCs System V (i.e. shmget e semget).

1.3.1 Identificadores

Os endereços da classe Protocol (i.e. Protocol::Address) permitem a multiplexação das NICs para suportar múltiplas sessões do protocolo e devem ser únicos neste sentido. Utilizar-se diretamente NIC::Address como Protocol::Physical_Address e os PIDs dos processos como Protocol::Port é tentador, pois a combinação gera números globalmente únicos e tem um regra de formação clara e de baixíssimo custo computacional. Entretanto, como uma VM poderia saber o PID de um processo rodando em outra VM para a comunicação entre sistemas?

A classe Communicator não usa endereçamento explícito, relegando qualquer forma de identificação de origem e destino das mensagens às próprias mensagens ou às outras entidades na pilha de comunicação. Todavia, como será visto na etapa 3, o recebedor de uma mensagem deve conseguir identificar as entidades necessárias para respondê-la.

1.3.2 Mensagens

Nesta etapa, as mensagens, além do array de bytes, agora denominado payload, devem incluir, ao menos, informações sobre a origem da mesma, de forma que possa ser respondida:

$M = \{\text{origin, payload}\}$

1.4 DESAFIO DA ETAPA 3 - Comunicação entre Sistemas Autônomos (e.g. Veículos)

Nesta etapa, a API deve ser implementada para suportar a comunicação entre sistemas autônomos, ou seja, entre as VMs que abstraem veículos, usando raw sockets para a transferência de frames Ethernet (sem IP). O principal ponto de interesse da etapa é a classe Engine referenciada na especificação da API e a programação imediata dos eventos de recepção de pacotes Ethernet pelo kernel para a pilha de protocolos.

1.4.1 Mensagens

Nesta etapa, as mensagens são um simples array de bytes encapsulados em um frame Ethernet

como payload:

$M = \{.*\}$

1.4.2 Identificadores

A plena identificação dos atores da comunicação é fundamental para o correto funcionamento da biblioteca sendo desenvolvida e pode ser feita de muitas formas diferentes. Os endereços na classe NIC (i.e. NIC::Address) são endereços Ethernet (a.k.a. MAC Address), e podem ser utilizados para identificar as VMs, mas é importante lembrar que uma mesma máquina, com mais de uma placa de rede, terá mais de um desses endereços. Além disso, é importante ter-se em mente que os requisitos do projeto exigem que o endereço destino na camada de enlace seja o de broadcast (FF:FF:FF:FF:FF:FF).

1.5 DESAFIO DA ETAPA 4 - Time-Triggered Publish-Subscribe Messages

Os agentes da comunicação, sejam componentes de um sistema autônomo, sejam sistemas autônomos se comunicando entre si, interagem através de mensagens de Interesse e de Resposta em um modelo similar ao publish-subscribe, mas sem publicações explícitas. Quando um agente quer um dado, ele envia uma mensagem de Interesse (todas as mensagens são enviadas em broadcast) que designa inequivocamente o tipo do dado no qual

ele tem interesse. Os agentes que recebem a mensagem de Interesse e que são capazes de produzir dados daquele tipo, enviam, periodicamente, mensagens de Resposta. Desta forma, cada mensagem deve carregar um código capaz de identificar sua natureza. Isto pode ser alcançado através dos códigos dos Transducer Electronic Data Sheet (TEDS) da norma IEEE 1451. Além disso, cada mensagem de Interesse deve especificar o período no qual os agentes devem enviar respostas e cada agente pertinente deve gerenciar o intervalo de forma correspondente. A comunicação orientada por eventos não está prevista para este projeto. Nesta etapa, a sincronização temporal das máquinas envolvidas na comunicação pode ser presumida.

As mensagens de interesse, portanto, devem ter a seguinte estrutura:

$I = \{\text{origin, type, period}\} // \text{period in us from now}$

Enquanto as de resposta:

$R = \{\text{origin, type, value}\}$

2. DESENVOLVIMENTO DA ETAPA 1

O grupo utilizou computador com sistema operacional Linux versão Ubuntu 24.04.3 LTS para desenvolvimento do trabalho e uso como computador host. Foi necessário configurar o kernel para rodar o riscv nas máquinas virtuais que seriam criadas futuramente, usando o comando de cross-compile.

Para inicializar as máquinas virtuais, foi utilizado o arquivo pré-compilado disponibilizado pelo professor, que sintetizou a preparação do ambiente. Para um teste inicial, duas máquinas virtuais foram inicializadas e testadas conexão direta através dos arquivos de send.cpp e receive.cpp. No projeto, essa ação de inicializar as máquinas virtuais e testar a conexão foi transferida para um arquivo shell que é chamado pelo makefile.

De modo geral, o projeto está estruturado como readme, makefile e run_vms.sh na pasta principal, pasta busybox e pasta src. Em src, encontramos todos os arquivos de cabeçalho para os objetos necessários ao projeto, assim como o código em C++ correspondente a esses. O principal arquivo é main.cpp, que é inicializado pelo init do busybox.

A seguir será apresentado uma pequena descrição de cada arquivo e partes do código, quando necessário.

2.1. Makefile e run_vms.sh

O Makefile automatiza o processo de compilação de um projeto em C++ para duas arquiteturas distintas: x86 e RISC-V. Ele define os compiladores específicos para cada caso, estabelece as opções de compilação e linkagem, organiza os arquivos-fonte e gera os binários correspondentes em diretórios separados. Após compilar o binário para RISC-V, ele gera um arquivo initramfs.cpio contendo o executável dentro da estrutura de diretórios da ferramenta BusyBox, possibilitando seu uso como sistema de arquivos inicial em máquinas virtuais. Assim, permite inicializar as máquinas virtuais necessárias para o projeto. Dentro do Makefile, o arquivo run_vms.sh é responsável por executar os comandos que criam a cada máquina virtual, que representarão os veículos. Cada veículo recebe um MAC Address diferente. Nessa etapa do projeto, serão simulados 5 veículos.

2.2. main.cpp e gateway.h

O arquivo main.cpp é o começo do projeto, onde é criado o processo principal que será representado pelo objeto Gateway. Esse processo será o canal de comunicação do veículo atual com outros veículos, além de comunicar também com os componentes internos. Por essa razão esse componente será chamado de gateway e será o único componente do veículo que terá uma placa de rede capaz de se comunicar com as outras via ethernet.

O gateway inicia seus parâmetros de placa de rede, protocolo e comunicador, que serão explorados nas próximas sessões. Além disso, o recurso de threads é usado para testar conexões, de modo que envia por broadcast por ethernet ou envia para locais por memória compartilhada. Por fim, o gateway fica disponível para receber comunicação.

```
class Gateway {
public:

    Gateway(){}

    int start() {
        try {
            EngineShm engShm;
            EngineEthernet engEth("eth0");
            Ethernet::Address myMac = engEth.mac();

            // NICs + Protocols
            NIC nicEth(&engEth, myMac);
            NIC nicShm(&engShm, myMac);
            Protocol<NIC> protoEth(&nicEth, ETYPE);
            Protocol<NIC> protoShm(&nicShm, ETYPE);

            // Communicator, route dst = local mac to shm and dst = broadcast to ethernet
            Communicator<NIC> comm(&protoEth, &protoShm, { myMac, PORT });
        }
    }
};
```

Figura 1 - Definições iniciais da classe Gateway

2.3. ethernet.h

Esta parte de código auxilia nos elementos necessários para Ethernet, definindo a classe Address, correspondente a um endereço MAC de 6 bytes, e a estrutura de um Frame, ou quadro de dados, composta dos endereços de origem e destino, do protocolo e da carga de dados propagados.

```
// Ethernet frame
struct Frame {
    Address dst;
    Address src;
    Protocol proto;
    static const int MAX_DATA = 1500;
    uint8_t data[MAX_DATA];
    unsigned int size; // real size of data

    Frame() : proto(0), size(0) {}
};
```

Figura 2 - Estrutura de um frame definida por ethernet.h

2.4. nic.h e nic.cpp

A classe NIC representa a placa de interface de rede do veículo, sendo vinculada a um Engine e contendo um endereço MAC. É por meio da NIC que um Engine envia dados em formato de Frame.

2.5. protocol.h

Um protocolo é composto por uma série de padrões que permitem realizar comunicação entre diferentes processos. Para isso, os objetos de protocolo precisam de NIC, Frame ProtocolNumber, Address e Port.

Para o correto funcionamento do projeto, é essencial que cada componente tenha identificador próprio, de modo que seja possível saber origem e destino da comunicação, independente de outros componentes. Para resolver isso, foi criado a estrutura Endpoint, tal que essa registra Mac Address e port únicos, e garante identificação de cada componente, conforme imagem abaixo.

```
// -----  
// Simple Protocol layered over NIC with logical ports.  
// - Endpoint = (MAC, port)  
// - Header = (srcPort, dstPort, length)  
// - Packet = Header + payload (bounded by Ethernet MTU)  
// - Async path: NIC notifies this Protocol via Observer<Frame,ProtocolNumber>.  
// - Sync path: optional receive() helper (not used when fully async).  
// -----  
template <typename TNIC>  
class Protocol : public Observer<NetFrame, NetProtocolType> {  
public:  
    using NIC      = TNIC;  
    using Frame    = NetFrame;  
    using ProtocolNumber = NetProtocolType;  
    using Address   = NetMacAddress;  
    using Port      = uint16_t;  
  
    struct Endpoint {  
        Address mac{};  
        Port    port{0};  
        Endpoint() = default;  
        Endpoint(Address a, Port p) : mac(a), port(p) {}  
        bool operator==(const Endpoint& o) const { return (mac == o.mac) && (port == o.port); }  
    };  
};
```

Figura 3 - Parte inicial da classe Protocol e estrutura Endpoint.

2.6. communicator.h e channel_origin.h

O communicator.h auxilia na implementação de funções que auxiliam na comunicação, como send, receive e try_receive. Essa classe também define uma estrutura que complementa a ideia do Endpoint, de modo que define que itens recebidos devem seguir a estrutura com origem, destino, payload e channel_origin. Essa última opera como uma flag para indicar se a comunicação é para componentes internos ou para outros veículos.


```

template <typename TNIC>
class Communicator {
public:
    using ProtocolT = Protocol<TNIC>;
    using Endpoint = typename ProtocolT::Endpoint;
    using Address = Ethernet::Address;

    struct Rx {
        Endpoint    from;
        Endpoint    to;
        std::vector<uint8_t> payload;
        ChannelOrigin origin;
    };
};

```

Figura 4 - Estrutura para receber comunicação.

2.7. Diagramas de sequência de recebimento de mensagem

Os diagramas facilitam o entendimento do fluxo do projeto. Para isso, o diagrama de sequência a seguir representa o caminho percorrido pela mensagem que está no barramento e chega até um gateway. No gateway, o pacote será recebido e seu frame será enviado para o objeto de destino. Assim, o protocolo será notificado e encaminha o pacote ao comunicador. Para encerrar a conexão, o comunicador envia essa mensagem no formato da estrutura rx para carComponent.

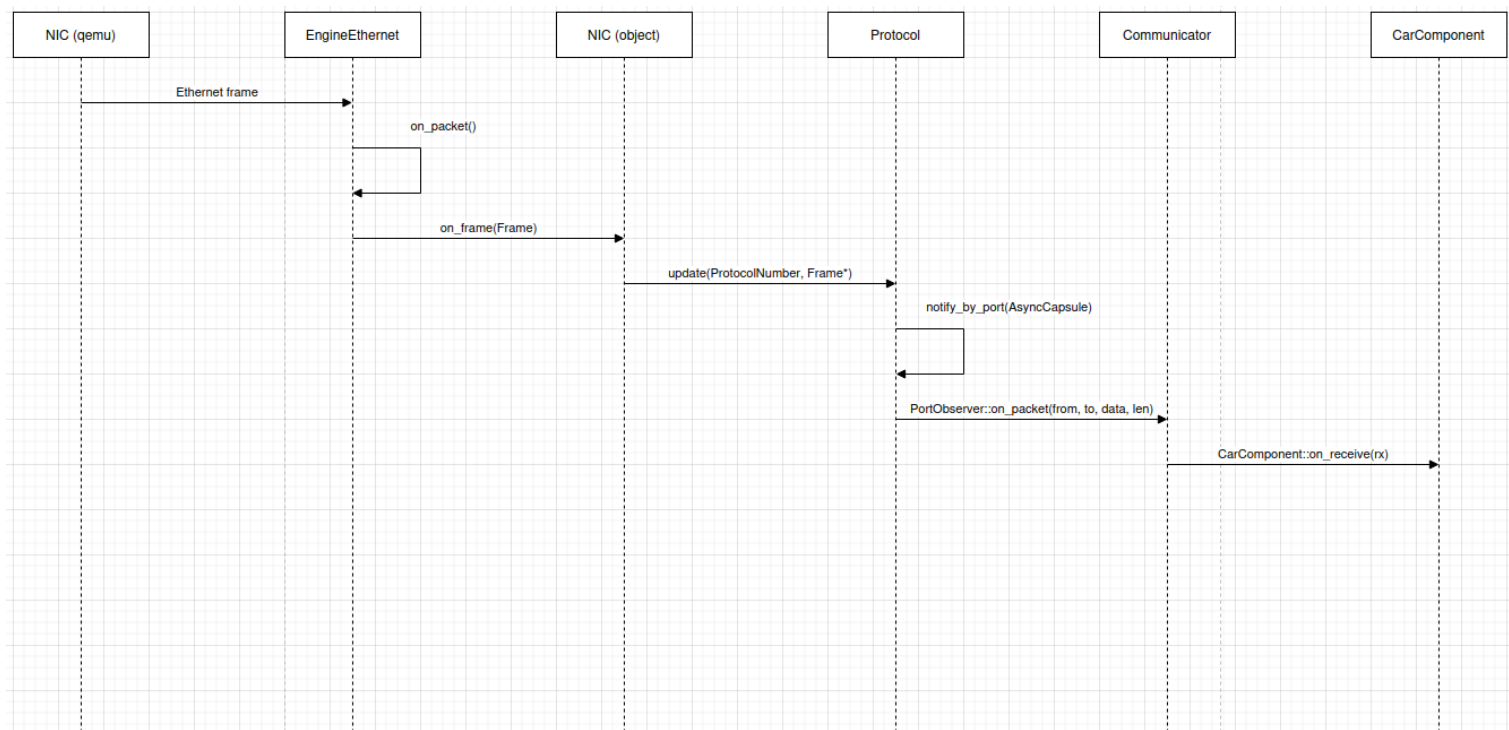


Figura 5 - Diagrama de sequência de recebimento de mensagem externa para componente interno do veículo

2.8. car_component.h e components.h

O arquivo `car_component` é quem cria os componentes do veículo e define as especificações gerais, como necessidade de protocolo, endpoint, endereço e comunicador. Para representar componentes, são usados novos processos, ou seja, comando `fork`. Para especificar o componente, podemos usar o `components.h`, que já dispõe alguns exemplos com funções básicas, como `PowertrainComponent` e `BrakeComponent`. Nessa classe foi possível testar a troca de informações do componente através de broadcast para outras máquinas virtuais, segundo imagem abaixo.

```
// ---- Powertrain ----
// Periodically publishes RPM to broadcast (Ethernet) and listens for control.

template <typename TNIC>
class PowertrainComponent : public CarComponent<TNIC> {
    using Base = CarComponent<TNIC>;
    using Rx = typename Base::CommunicatorT::Rx;
public:
    PowertrainComponent(typename Base::CommunicatorT* comm, uint16_t port)
        : Base(comm, port, "Powertrain") {}

protected:
    bool wants_tick() const override { return true; }
    unsigned tick_period_ms() const override { return 2000; } // every 2s

    void on_tick() override {
        _rpm = (_rpm + 300) % 6000;
        char msg[64];
        std::snprintf(msg, sizeof(msg), "rpm=%u torque=%u", _rpm, 250u);
        Base::send_broadcast(msg, std::strlen(msg));
    }

    void on_receive(const Rx& rx) override {
        // Example: accept commands addressed to this port (from gateway or others)
        auto s = Base::to_string(rx.payload);
    }

private:
    unsigned _rpm{900};
};
```

Figura 6 - Exemplo de componente específico com troca de informações

2.9. engine.h, engine_ethernet.h e engine_shm.h

Para as definições da rede, a classe abstrata `engine` coloca as funções `send` e `star` como fundamentais para esse objeto. As especificações de `engine_ethernet` envolvem a parte de socket informações de Mac Address, enquanto `engine_shm` retem informações do processo específico. A troca de mensagens por memória compartilhada será foco da segunda entrega do projeto e a troca de mensagens por ethernet será foco da terceira entrega do projeto.

2.10. Testes de comunicação entre duas máquinas virtuais

A imagem a seguir mostra comunicação bem sucedida entre duas máquinas virtuais, que representam veículos.

Figura 7 - Teste bem sucedido de comunicação entre máquinas virtuais

2.11. Avaliação simples de desempenho

O tempo entre envio de mensagem e recebimento de mensagem broadcast foi obtido através de logs em testes, sendo esse identificado como em média 50ms, conforme imagem abaixo.

Figura 8 - Avaliação simples de desempenho

3. DESENVOLVIMENTO DA ETAPA 2

A etapa 2 consistiu na implementação de comunicações entre os componentes de um veículo por memória compartilhada. Cada componente possui um NIC, um Communicator e um Protocol para receber mensagens tanto de memória compartilhada como de broadcast. Além disso, foi implementada uma barreira para sincronizar a inicialização de cada veículo e foram corrigidos os erros da primeira entrega.

A seguir, são descritas as novas classes criadas para a etapa, além das classes modificadas após a avaliação da etapa 1.

3.1. gateway.h

A classe Gateway foi modificada para ser uma subclasse de CarComponent, agora sendo somente responsável por enviar a mensagem “READY” a outras máquinas virtuais instanciadas, adicionar as mensagens “READY” recebidas a uma lista e, quando receber de todas as outras máquinas, enviar a mensagem “GO”, que sinaliza que a sincronização entre as máquinas foi concluída. Os componentes das máquinas virtuais só começam a ser criados depois que todas as máquinas iniciaram.

3.2. car.h e car.cpp

A classe Car desempenha o papel antes cumprido pela classe Gateway na etapa 1, sendo uma representação de um veículo que reúne todos os diferentes CarComponents e os inicializa para estabelecer as comunicações internas.

```
13  void Car::start()
14  {
15      int components_len = this->components.size();
16      Gateway<NIC>* gateway = new Gateway<NIC>(PORT);
17
18      gateway->initialize(true, components_len);
19
20      for (int i = 0; i < components_len; i++)
21      {
22          int pid = fork();
23          if(pid == 0)
24          {
25              CarComponent<NIC>* comp = this->components.at(i);
26              comp->initialize(false, components_len);
27              return;
28          }
29      }
30
31      gateway->on_tick_loop();
32  }
```

Figura 9 - Definição do método “start” da classe Car.

3.3. shm_node.h e shm_node.cpp

A classe SHM_Node implementa a lógica de um nó de comunicação usando memória compartilhada e mecanismos de sincronização entre processos usando POSIX threads (pthreads). O objetivo desse código é permitir que múltiplos processos (ou nós) troquem mensagens entre si de forma eficiente e sincronizada, utilizando uma região de memória compartilhada e primitivas de sincronização.

4. DESENVOLVIMENTO DA ETAPA 3

A equipe iniciou corrigindo itens da entrega anterior. Agora apenas o componente Gateway terá a NIC para comunicação com as outras máquinas virtuais.

A parte de sincronização para inicialização das máquinas virtuais passou a acontecer na classe protocol. Os componentes passam a receber um número de porta específico. Como nesse caso ainda temos apenas um componente instanciado, apenas uma porta foi exemplificada. Nas próximas entregas, espera-se estabelecer um padrão de valores para portas de componentes, conforme o tipo e ordem de criação.

```
src > car.cpp > start()

1  #include "car.h"
2
3  Car::Car()
4  {
5      this->components.push_back(new PowertrainComponent<NIC>(80));
6      // each component receive a different port
7  }
8
```

Figura 10 - Cada componente recebe uma porta específica.

Para o makefile, foram realizadas trocas de comandos, assim evitando que o busybox compile a cada chamada.

4.1 Sinal da Kernel

Na entrega anterior, já estávamos utilizando sinais da kernel para informar aos componentes sobre o recebimento de mensagens. No entanto, o signal handler estava executando métodos da camada de aplicação, o que não é signal-safe. Modificamos a implementação para que o signal handler apenas realize um *post* em um semáforo, enquanto que uma thread de leitura drena o *buffer* e repassa a informação para as camadas superiores da *stack*.

```
130 // SIGIO
131 void EngineEthernet::sigio_handler(int) {
132     //Async-signal-safe: just post the semaphore
133     sem_post(&packet_sem);
134 }
135
136 // Receive Loop
137 void EngineEthernet::rx_loop() {
138     while (running) {
139         // This blocks until the semaphore is posted by SIGIO
140         sem_wait(&packet_sem);
141
142         if (!running) break;
143
144         on_packet(); // Drain all packets in the socket buffer
145     }
146 }
147
```

Figura 11 - Parte do código de engine_ethernet.cpp

```
148 // Drain all available frames and forward to NIC
149 void EngineEthernet::on_packet()
150 {
151     if(sock_ < 0) return;
152
153     while(true)
154     {
155         uint8_t buf[ETH_FRAME_LEN]; // ~1514 bytes
156         ssize_t n = ::recvfrom(sock_, buf, sizeof(buf), 0, nullptr, nullptr);
157         if(n < 0) break; //EAGAIN when drained
158
159         if(static_cast<size_t>(n) < sizeof(ether_header)) continue;
160     }
161 }
```

Figura 12 - Parte do código de engine_ethernet.cpp

5. DESENVOLVIMENTO DA ETAPA 4

A equipe iniciou essa fase adaptando o arquivo de makefile para compilação com x86, assim evitando a compilação cruzada com RISC-V. Além disso, o assunto de publish-subscribe exigiu tempo de estudo para entender onde seria necessário adaptar o projeto

A ideia de publish-subscribe, desafio dessa entrega, foi incluída em cada componente do carro. As informações de inscrição foram adicionadas no payload. E para gerenciar as capacidades, uma nova classe `publish_subscriber` é utilizada.

```
static bool digest_subs(const Rx& rx) {
    std::string s(rx.payload.begin(), rx.payload.end());
    size_t p1 = s.find("SUBS=");
    size_t p2 = s.find("SUBS_TYPE=");
    size_t p3 = s.find("SUBS_PERIOD=");
    if (p1 == std::string::npos || p2 == std::string::npos || p3 == std::string::npos) return false;

    size_t e1 = s.find(' ', p1); if (e1 == std::string::npos) e1 = s.size();
    size_t e2 = s.find(' ', p2); if (e2 == std::string::npos) e2 = s.size();
    size_t e3 = s.find(' ', p3); if (e3 == std::string::npos) e3 = s.size();

    std::string addr = s.substr(p1 + 5, e1 - (p1 + 5));
    VehicleDataType dtype = std::stoi(s.substr(p2 + 10, e2 - (p2 + 10)));
    u64 period = std::stoull(s.substr(p3 + 12, e3 - (p3 + 12)));

    if(this->_pubSub.supports(dtype)) this->_pubSub.subscribe(dtype, addr, period);

    return true;
}
```

Figura 13 - Ajustes no payload.

Por fim, foram realizadas algumas melhorias no projeto, como inclusão de gerador de logs e registro de latência através de script em Python em arquivos externos. As máquinas virtuais também foram ajustadas para encerrar, sendo isso determinado por timeout no makefile.

6. CONCLUSÃO

6.1. Dificuldades anteriores

O primeiro desafio da equipe foi compreender a arquitetura esperada para o projeto e, principalmente, relacionar as comunicações que deveriam ser entregues nesta etapa. Além disso, a fusão de conteúdos de diversas disciplinas fez com que a análise e entendimentos dos códigos base fornecidos demorasse mais tempo do que o esperado. Algumas decisões de projeto também representaram desafios por ainda não conhecermos todas as etapas que precisam ser implementadas e o impacto da escolha nesse momento.

De forma geral, a equipe conseguiu concluir as tarefas propostas para a primeira entrega do projeto de Comunicação em Sistemas Autônomos Críticos. Foi possível configurar o ambiente, inicializar as máquinas virtuais, que representam os veículos, simular comunicação

broadcast, montar a estrutura geral da API, avaliar latência de comunicação e concluir os entregáveis para essa etapa. Nessa entrega não foi possível simular a comunicação da memória compartilhada.

Durante a realização dessa etapa foi possível perceber que algumas otimizações podem ser aplicadas para as próximas etapas, como execução das máquinas virtuais em diferentes terminais, alterar o tipo da mensagem do payload para classe message e ampliar a funcionalidade dos componentes implementados.

Nas próximas etapas espera-se implementar a parte de troca de mensagens entre componentes internos do veículo, com memória compartilhada. E, em seguida, implementar a comunicação entre veículos diferentes.

Durante a realização da etapa dois, o grupo passou boa parte do tempo identificando problemas relacionados ao Makefile e a inicialização das máquinas virtuais, corrigindo erros que estavam causando falha de sobrescrição de memória e erro de compilação na execução. A solução final que rodou no computador de todos integrantes do grupo foi criar a pasta busybox em cada execução.

A primeira abordagem da equipe para a etapa dois foi resolver os problemas apontados pelos comentários do professor na entrega anterior. A definição de componentes foi melhorada com a inserção do objeto “car”, de modo que o gateway passou a ser um componente. Os protocolos foram unificados para a NIC. Ao iniciar as máquinas virtuais, cada uma instancia o componente gateway e passa por uma barreira, de modo que outros componentes só são criados depois. Por fim, a equipe trabalhou no desenvolvimento da memória compartilhada entre os componentes.

Os ajustes realizados durante essa etapa foram essenciais para o bom rendimento do projeto, como por exemplo rodar no computador de todos os integrantes da equipe. Para próxima entrega espera-se seguir com os desafios propostos.

6.2. Conclusão final após etapa 4

De forma geral, a equipe conseguiu concluir as tarefas propostas para a quarta entrega do projeto de Comunicação em Sistemas Autônomos Críticos. Foi possível implementar publish-subscribe, além de aplicar melhorias no projeto como um todo.